COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	August 20, 2026	**DATE SUBMITTED:**	August 20, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Assignment 6.1 **Linear and Binary Search**

1. PROCEDURE OUTPUT

What procedures did you perform?

1. What is a search tree in data structures?

The **search tree** is a non-linear tree data structure which is structured in a hierarchical manner and used to store keys in a particular sorted manner for quick searching. A **Binary Search Tree (BST)** has each node with at most two children and the search properties are defined as:

- All keys of the **left subtree** must be smaller than the root/parent node.
- All keys of the **right subtree** must be larger than the root/parent node.

2. Different Types of Search Algorithms & Comparison

- **Linear / Sequential Search**: Goes through each item from index 0 to n-1 until the matching item is encountered or until the end of the list is reached. Linear / sequential search can be done with both unsorted and sorted data structures (such as arrays and linked lists).
- **Binary Search (Divide and Conquer)**: Consists of dividing the already sorted array in half in each step by comparing the key to the middle element of the array.
- **Jump / Interpolation Search**: Types of Interval search for a sorted array that jumps through the array using fixed jumps or estimates the probable location of the key based on its value.
- **Hashing Search**: Makes use of a hashing function to directly access a key in a bucket array (Hash Table).
- **Tree and Graph Search (DFS / BFS)**: Searching a non-linear data structure by traversing through paths in the graph/tree.

3. Operations & Implementations Performed Using Linear and Binary Search

- **Element Lookup & Validation of its Presence**: Checking for the presence of an element in the collection (contains / find).
- **Accessing Index**: Fetching the precise index or pointer of the record.
- **Finding Duplicates & Boundary Values**: Detecting the initial, final, or total number of duplicate elements (lower_bound / upper_bound using binary search).
- **Search in Unsorted or Linked Lists**: Linear search is used for dynamically growing singly / doubly linked lists which cannot be accessed randomly.
- **Threshold and Peak Value Finding**: Binary search is used extensively over monotonic mathematical intervals or sorted structures.

4. Advantages of Using a Binary Search Tree (BST)

- **Dynamic Allocation**: Nodes of a BST can be allocated dynamically, as there is no restriction like static arrays.
- **Sorting**: Using in-order traversal (Left - > Root -> Right), one can obtain all the stored elements in the sorted order, and this does not require any external sort.
- **Searching & Deletion**: Searching and dynamic insertion and deletion can be performed simultaneously, and there is no need to shift the elements as in case of sorted arrays.
- **Range and Nearest Neighbor**: Min and Max, as well as predecessor and successor, can easily be obtained.

5. Example C++ Program (Linear & Binary Search)

```C/C++
#include <iostream>


// Linear Search Implementation
int linearSearch(const int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Target found at index i
        }
    }
    return -1; // Target not found
}


// Binary Search Implementation (Requires Sorted Array)
int binarySearch(const int arr[], int size, int target) {
    int low = 0;
```

```
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid; // Target found
        }
        else if (arr[mid] < target) {
            low = mid + 1; // Search right half
        }
        else {
            high = mid - 1; // Search left half
        }
    }
    return -1; // Target not found
}

int main() {
    // Sample sorted array
    int data[] = {12, 24, 35, 47, 53, 68, 79, 90};
```

```
    int size = sizeof(data) / sizeof(data[0]);
    int target = 53;

    std::cout << "--- SEARCH ALGORITHM DEMONSTRATION ---\n";
    std::cout << "Target Value: " << target << "\n\n";

    // 1. Testing Linear Search
    int linResult = linearSearch(data, size, target);
    if (linResult != -1) {
        std::cout << "[Linear Search] Found at index: " <<
linResult << "\n";
    } else {
        std::cout << "[Linear Search] Element not found.\n";
    }

    // 2. Testing Binary Search
    int binResult = binarySearch(data, size, target);
    if (binResult != -1) {
        std::cout << "[Binary Search] Found at index: " <<
binResult << "\n";
    } else {
        std::cout << "[Binary Search] Element not found.\n";
```

```
    }
    return 0;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **I used AI to correct my grammar mistakes iny my answers and explanation of concepts in formulating my code in number 5 item.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I just used AI so that I can precisely express my answers in every item. I also used AI so that I can understand the concept of Linary and Binary Search because we still not able to discuss it.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.